

第3章

关系数据库标准语言 SQL

《概论》第3章详细介绍了结构化查询语言（SQL）。SQL 是关系数据库的标准语言，其内容十分丰富，是关系数据库概念和技术的重要组成部分。

3.1 基本知识点

关系模型和关系数据库是《概论》的重点，第3章则是重点中的重点，因为关系数据库系统的主要功能都是通过 SQL 来实现的。

① 需要了解的：了解 SQL 的发展过程，从而进一步了解关系数据库技术和关系数据库管理系统（RDBMS）产品的发展过程。

② 需要牢固掌握的：掌握 SQL 的特点和优点，体会面向过程的语言与 SQL 的区别；体会关系数据库系统为数据库应用系统的开发提供良好的环境、减轻用户的负担、提高用户生产率的原因。

③ 需要举一反三的：熟练且正确使用 SQL 完成对数据库的查询、插入、删除、更新操作，特别要熟练掌握 SQL 强大的查询功能。

在完成具体的 SQL 语句时，希望读者能有意识地 and 关系代数、关系演算等语言进行比较，了解其各自的特点。

④ 难点：本章的难点在于用 SQL 正确地完成复杂查询。因此，读者在学习的过程中一定要多加练习，要在一种 RDBMS 产品上进行实际运行，以检查自己的答案是否正确。只有通过大量练习，才能真正达到举一反三的熟练程度。

3.2 习题解答和解析

1. 试述 SQL 的特点。

【解析】详细内容可参考《概论》第3章 3.1.2 小节相关内容。请读者注意不仅要知道这些特点，更关键的是要通过使用 SQL 语句进行具体的练习来更好地理解这些特点。

答：

① 功能综合且风格统一。SQL 集数据定义语言（DDL）、数据操纵语言（DML）、数据控

制语言 (DCL) 的功能于一体。

② 数据操纵高度非过程化。用 SQL 进行数据操作时, 只要提出“做什么”而不必指明“怎么做”, 因此无须了解存取路径。存取路径的选择以及 SQL 语句的操作过程都是由系统自动完成的。

③ 面向集合的操作方式。SQL 采用集合操作方式, 不仅操作对象、查询结果可以是元组的集合, 而且一次插入、删除、更新操作的对象也可以是元组的集合。

④ 以统一的语法结构提供多种使用方式。SQL 既是独立的语言, 又是嵌入式语言。作为独立的语言, SQL 能够独立地以联机交互方式使用; 作为嵌入式语言, SQL 语句能够嵌入高级语言 (如 C、C++、Java、Python) 程序, 供程序员设计程序时使用。

⑤ 语言简洁且易学易用。

*2. 说明在 DROP TABLE 时, RESTRICT 和 CASCADE 的区别。

答: RESTRICT 表示该表的删除是有限制条件的。例如, 要删除的基本表不能被其他表的约束所引用, 不能有视图, 不能有触发器, 不能有存储过程或函数等。如果存在这些依赖该表的对象, 则该表不能被删除。

CASCADE 表示该表的删除没有限制条件, 在删除基本表的同时, 相关的依赖对象 (如视图) 都将被删除。

【解析】不同的数据库产品, 在具体实现和处理 DROP TABLE 的策略上, 会与 SQL 标准有所差别。我们对比了 DROP TABLE 的 SQL 标准与 Kingbase ES、Oracle、MS SQL Server 三种数据库产品的不同处理策略。读者可以扫描《概论》第 3 章 3.2.2 小节中的二维码, 阅读“不同产品 DROP TABLE 处理策略比较”中的内容。

3. 有两个关系 $S(A, B, C, D)$ 和 $T(C, D, E, F)$, 写出与下列查询等价的 SQL 表达式:

- ① $\sigma_{A=10}(S)$ ② $\pi_{A,B}(S)$ ③ $S \bowtie T$
 ④ $S \bowtie_{S.C=T.C} T$ ⑤ $S \bowtie_{A<E} T$ ⑥ $\pi_{C,D}(S) \times T$

答:

- ① SELECT * FROM S WHERE A = 10
 ② SELECT DISTINCT A, B FROM S
 ③ SELECT A, B, S.C, S.D, E, F FROM S, T WHERE S.C = T.C AND S.D = T.D

【解析】这是自然连接。

- ④ SELECT A, B, S.C, S.D, T.C, T.D, E, F FROM S, T WHERE S.C = T.C

【解析】这是等值连接。

- ⑤ SELECT A, B, S.C, S.D, T.C, T.D, E, F FROM S, T WHERE A < E

- ⑥ SELECT S1.C, S1.D, T.C, T.D, E, F FROM (SELECT DISTINCT(C, D) FROM S) AS S1, T

【解析】(SELECT DISTINCT C, D FROM S) AS S1: S 表在属性列 C 和 D 上投影, 从结果集中去掉重复行后把结果集命名为 S1, 然后连接 S1 和 T 表, 生成 S1 和 T 的笛卡儿积。

4. 用 SQL 语句建立第 2 章习题 6 中 SPJ 数据库的 4 个表 S、P、J 及 SPJ, 针对建立的表用 SQL 完成第 2 章习题 6 中的查询。

答: 建 S 表:

```
S(SNO, SNAME, STATUS, CITY);  
CREATE TABLE S  
    (SNO CHAR(3) PRIMARY KEY,  
     SNAME CHAR(10),  
     STATUS CHAR(2),  
     CITY CHAR(10));
```

建 P 表:

```
P(PNO, PNAME, COLOR, WEIGHT);  
CREATE TABLE P  
    (PNO CHAR(3) PRIMARY KEY,  
     PNAME CHAR(10),  
     COLOR CHAR(4),  
     WEIGHT INT);
```

建 J 表:

```
J(JNO, JNAME, CITY);  
CREATE TABLE J  
    (JNO CHAR(3) PRIMARY KEY,  
     JNAME CHAR(10),  
     CITY CHAR(10));
```

建 SPJ 表:

```
SPJ(SNO, PNO, JNO, QTY);  
CREATE TABLE SPJ  
    (SNO CHAR(3),  
     PNO CHAR(3),  
     JNO CHAR(3),  
     QTY INT,  
     PRIMARY KEY (SNO,PNO,JNO);           /*表级完整性定义, 主码由三个属性构成*/  
     FOREIGN KEY (SNO) REFERENCES S(SNO); /*参照完整性定义*/  
     FOREIGN KEY (PNO) REFERENCES P(PNO); /*参照完整性定义*/  
     FOREIGN KEY (JNO) REFERENCES J(JNO)); /*参照完整性定义*/
```

【解析】读者完成建表后, 应先插入若干数据 (如第 2 章习题 6)。

- ① 求供应工程项目 J1 零件的供应商代码 SNO。

```
SELECT SNO
FROM SPJ
WHERE JNO='J1';
```

- ② 求供应工程项目 J1 零件 P1 的供应商代码 SNO。

```
SELECT SNO
FROM SPJ
WHERE JNO='J1' AND PNO='P1';
```

- ③ 求供应工程项目 J1 红色零件的供应商代码 SNO。

```
SELECT SNO                                /*这是嵌套查询*/
FROM SPJ
WHERE JNO='J1' AND PNO IN                 /*找出红色零件的零件代码 PNO */
    (SELECT PNO
     FROM P                               /*从 P 表中找红色零件*/
     WHERE COLOR='红');
```

或

```
SELECT SNO
FROM SPJ, P                               /*这是两表连接*/
WHERE SPJ.JNO='J1' AND SPJ.PNO=P.PNO AND P.COLOR='红';
/*这是复合条件连接查询*/
```

- ④ 求没有使用天津供应商生产的红色零件的工程项目代码 JNO。

【解析】读者可以对比第2章习题6第④题中用 ALPHA 语言完成该查询的解答：
ALPHA 语言：

```
RANGE SPJ SPJX
      P PX
      S SX
GET W (J.JNO): ¬∃SPJX(SPJX.JNO=J.JNO ∧
    ∃SX(SX.SNO=SPJX.SNO ∧ SX.CITY='天津' ∧
    ∃PX(PX.PNO=SPJX.PNO ∧ PX.COLOR='红'))
```

如果理解了有关该题的解析说明，那么本题的解答可以看成是把关系演算用 SQL 来表示的过程。

```
SELECT JNO    /*这种解法是使用多重嵌套查询*/
FROM J        /*注意：从 J 表入手，以包含那些尚未使用任何零件的工程项目代码*/
```



```

WHERE NOT EXISTS
  (SELECT *
   FROM SPJ
   WHERE SPJ.JNO=J.JNO AND SNO IN
        (SELECT SNO /*天津供应商的 SNO*/
         FROM S
         WHERE CITY='天津')
   AND PNO IN /*红色零件的 PNO*/
        (SELECT PNO
         FROM P
         WHERE COLOR='红') );

```

或

```

SELECT JNO
FROM J
WHERE NOT EXISTS
  (SELECT *
   FROM SPJ, S, P /*这里的子查询是一个多表连接*/
   WHERE SPJ.JNO=J.JNO AND SPJ.SNO=S.SNO
   AND SPJ.PNO=P.PNO AND S.CITY='天津'
   AND P.COLOR='红');

```

⑤ 求至少使用了与供应商 S1 所供应的全部零件相同零件代码的工程项目代码 JNO。

【解析】请阅读《概论》第 3 章 3.3.3 小节[例 3.65]。

本查询可以抽象为：/*工程项目 x ，零件 y */

要求这样的工程项目 x ，使 $(\forall y)p \rightarrow q$ 为真。即：

对于所有的零件 y ，满足逻辑蕴涵 $p \rightarrow q$ ：

p 表示谓词“供应商 S1 供应了零件 y ”。

q 表示谓词“工程项目 x 选用了零件 y ”。

即：只要“供应商 S1 供应了零件 y ”为真，则“工程项目 x 选用了零件 y ”为真。

逻辑蕴涵可以转换为等价形式：

$$(\forall y)p \rightarrow q \equiv \neg (\exists y (\neg (p \rightarrow q))) \equiv \neg (\exists y (\neg (\neg p \vee q))) \equiv \neg \exists y (p \wedge \neg q)$$

它所表达的语义为：不存在这样的零件 y ，供应商 S1 供应了 y ，而工程项目 x 没有选用 y 。

用 SQL 语言表示如下：

```

SELECT DISTINCT JNO
FROM SPJ SPJZ
WHERE NOT EXISTS /*这是一个相关子查询 */
  (SELECT * /*父查询和子查询均引用了 SPJ 表*/
   FROM SPJ SPJX /*用别名 SPJZ、SPJX 将父查询*/

```

```
WHERE SNO='S1'           /*与子查询中的 SPJ 表区分开*/
AND NOT EXISTS
(SELECT *                 /*用别名 SPJY 与父查询*/
FROM SPJ SPJY            /*中的 SPJ 表区分开*/
WHERE SPJY.PNO=SPJX.PNO
AND SPJY.JNO=SPJZ.JNO));
```

可以把 SQL 和关系代数、ALPHA 语言、QBE 语言进行比较，体会各种语言的优点。

5. 针对上面第4题中的4个表 S、P、J 及 SPJ，试用 SQL 语句完成以下各项操作。

答：

- ① 找出所有供应商的姓名和所在城市。

```
SELECT SNAME, CITY
FROM S;
```

- ② 找出所有零件的名称、颜色、重量。

```
SELECT PNAME, COLOR, WEIGHT
FROM P;
```

- ③ 找出使用供应商 S1 所供应零件的工程项目代码。

```
SELECT JNO
FROM SPJ
WHERE SNO='S1';
```

- ④ 找出工程项目 J2 使用的各种零件的名称及其数量。

```
SELECT P.PNAME, SPJ.QTY
FROM P, SPJ
WHERE P.PNO=SPJ.PNO AND SPJ.JNO='J2';
```

- ⑤ 找出上海厂商供应的所有零件代码。

```
SELECT DISTINCT PNO
FROM SPJ
WHERE SNO IN
  (SELECT SNO
   FROM S
   WHERE CITY='上海');
```

- ⑥ 找出使用上海产的零件的工程名称。

```
SELECT JNAME
FROM J, SPJ, S
```

```
WHERE J. JNO=SPJ. JNO AND SPJ. SNO=S.SNO AND S.CITY='上海';
```

或

```
SELECT JNAME  
FROM J  
WHERE JNO IN  
    (SELECT JNO  
     FROM SPJ, S  
     WHERE SPJ. SNO=S.SNO AND S.CITY='上海');
```

- ⑦ 找出没有使用天津产的零件的工程项目代码。

```
SELECT JNO  
FROM J  
WHERE NOT EXISTS  
    (SELECT *  
     FROM SPJ  
     WHERE SPJ.JNO=J.JNO AND SPJ. SNO IN  
         (SELECT SNO  
          FROM S  
          WHERE CITY='天津'));
```

或

```
SELECT JNO  
FROM J  
WHERE NOT EXISTS  
    (SELECT *  
     FROM SPJ, S  
     WHERE SPJ.JNO=J.JNO AND SPJ.SNO=S.SNO  
         AND S.CITY='天津');
```

- ⑧ 把全部红色零件的颜色改成蓝色。

```
UPDATE P  
SET COLOR='蓝'  
WHERE COLOR='红';
```

- ⑨ 由 S5 供给 J4 的零件 P6 改为由 S3 供应 (请做必要的修改)。

```
UPDATE SPJ  
SET SNO='S3'  
WHERE SNO='S5' AND JNO='J4' AND PNO='P6';
```

- ⑩ 从供应商关系中删除 S2 的记录，并从供应情况关系中删除相应的记录。

```
DELETE
FROM SPJ
WHERE SNO='S2';
DELETE
FROM S
WHERE SNO='S2';
```

- ⑪ 将 (S2, J6, P4, 200) 插入供应情况关系。

```
INSERT INTO SPJ(SNO, JNO, PNO, QTY)      /*INTO 子句中指明列名*/
VALUES (S2, J6, P4, 200);                 /*插入的属性值与指明列要对应*/
```

或

```
INSERT INTO SPJ                                /*INTO 子句中没有指明列名*/
VALUES (S2, P4, J6, 200);                     /*插入的记录在每个属性列上均有值*/
                                              /*并且属性列要和表定义中的次序一致*/
```

6. 什么是基本表？什么是视图？两者的区别和联系是什么？

答：基本表是本身独立存在的表，在 SQL 中一个关系就对应一个基本表。

视图是从一个或几个基本表导出的表。视图本身不独立存储在数据库中，是一个虚表，即数据库中只存放视图的定义而不存放视图对应的数据，这些数据仍存放在导出该视图的基本表中。视图在概念上与基本表等同，用户可以如同使用基本表那样使用视图，可以在视图上再定义新的视图。

7. 试述视图的优点。

答：请阅读《概论》第3章 3.6.4 小节相关内容。

- ① 视图能够对机密数据提供安全保护。
- ② 视图对重构数据库提供了一定程度的逻辑独立性。
- ③ 视图能够简化用户的操作。
- ④ 视图使用户能以多种角度看待同一数据。

8. 哪类视图是可以更新的？哪类视图是不可更新的？各举一例说明。

答：请阅读《概论》第3章 3.6.3 小节相关内容和示例。

基本表的行列子集视图一般是可更新的。若视图的属性来自聚集函数、表达式，则该视图肯定是不可更新的。

【解析】首先要区分两个概念：“不可更新的视图”与“不允许更新的视图”。前者指理论上已证明其是不可更新的，后者与各数据库管理系统产品的规定有关。不同产品对视图的更新规定不尽相同，读者应参考具体产品手册，正确写出对视图的更新语句。

9. 针对第2章习题6建立的4个表, 请为三建工程项目建立一个供应情况的视图, 包括供应商代码(SNO)、零件代码(PNO)、供应数量(QTY)。针对该视图完成下列查询:

- ① 找出三建工程项目使用的各种零件代码及其数量。
- ② 找出供应商 S1 供应三建工程的情况。

答: 创建视图:

```
CREATE VIEW V_SPJ AS
SELECT SNO, PNO, QTY
FROM SPJ
WHERE JNO=
    (SELECT JNO
     FROM J
     WHERE JNAME='三建');
```

对该视图查询:

```
SELECT PNO, QTY
FROM V_SPJ;
```

- ③ SELECT PNO, QTY /* S1 供应三建工程的零件代码和对应的数量*/

```
FROM V_SPJ
WHERE SNO='S1';
```

10. 什么是空值? 请举例说明。SQL 中如何表示空值? 空值如何参与运算?

【解析】请阅读《概论》第3章3.5节“空值的处理”相关内容。

空值是实际场景下存在的一种不确定的值, 即目前“不知道”“不存在”或“无意义”的值。例如, 将学生信息插入 Student 表时, 有些学生的信息漏报了, 这些属性就取空值。

数据库用 NULL 表示空值。在创建基本表时, 如果属性定义(或者域定义)为 NOT NULL 约束, 则该属性不能取空值。空值与另一个值(包括另一个空值)的算术运算的结果为空值。空值与另一个值(包括另一个空值)的比较运算结果为 UNKNOWN。

3.3 补充习题

1. 选择题

- ① 关于 SQL, 下列说法正确的是 ()。
- A. 数据控制功能不是 SQL 的功能之一
 - B. SQL 采用的是面向记录的操作方式, 以记录为单位进行操作
 - C. SQL 是非过程化的语言, 用户无须指定存取路径
 - D. SQL 作为嵌入式语言, 其语法与独立的语言有较大差别